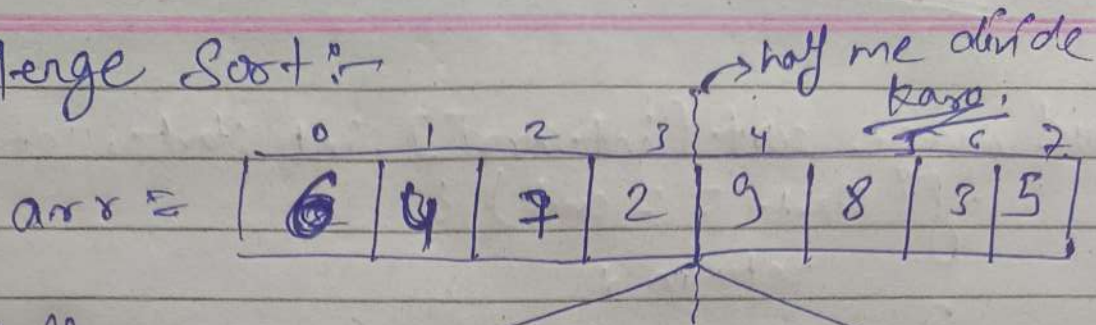
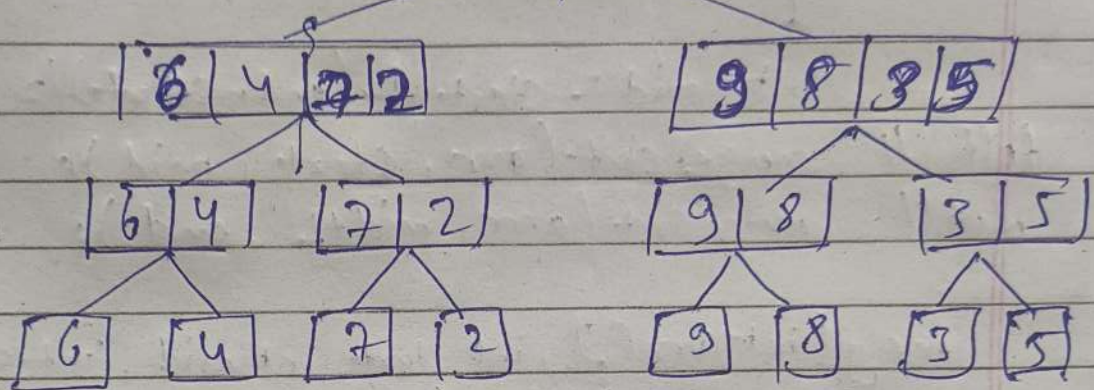


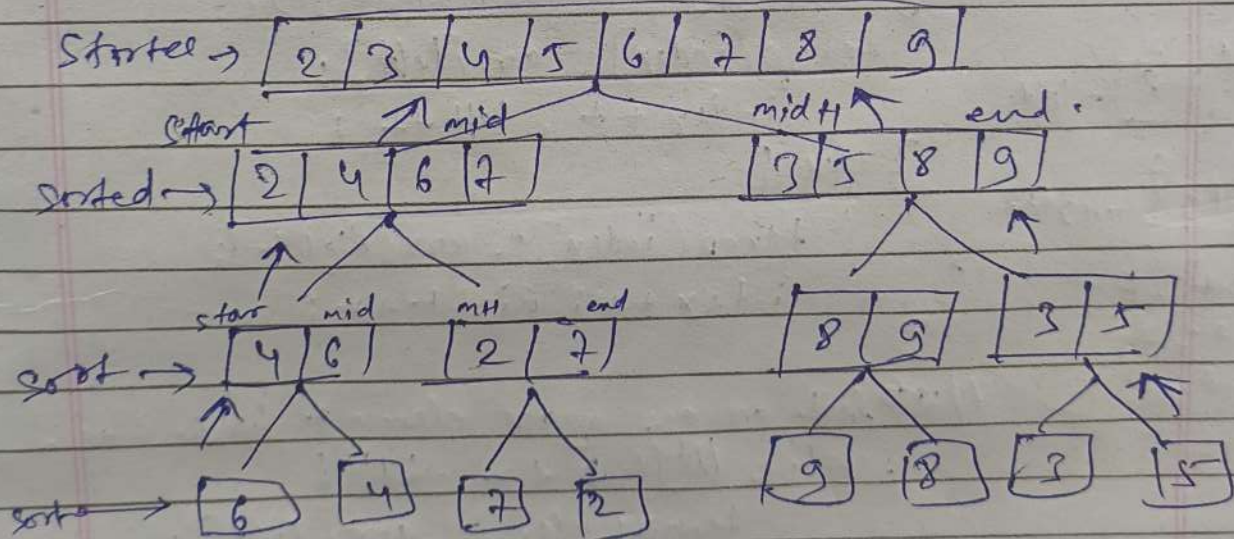
A. Merge Sort :-



half-half me divide krte jana jabtak single element na bache.



→ Ab inche se sabko sort or arrange krte merge krte hue uper jao.



Left = start, mid

Right = mid+1, end.

Array ko do partion me krto ek left or ek right jab tak single element nhi bach gae, i.e. (start == end) nhi ho gata.

Pseudo code →

```
void merge_sort (int arr[], int start, int end) {
    if (start == end) {
        return;
    }
```

```
    int mid = start + (end - start) / 2;
    left // merge_sort(arr, start, mid);
    right // merge_sort(arr, mid + 1, end);
```

```
    merge (int arr[], int start, int mid, int end) {
        vector<int> temp (end - start + 1);
```

index
↓

1	2	3	4	5
---	---	---	---	---

1	3	4
---	---	---

← left

2	5
---	---

← right

start = 5

mid = 2

end = 5

```
    int left = start, right = mid + 1, index = 0;
    while (left <= mid & right <= end) {
```

```
        if (arr[left] <= arr[right]) {
```

```
            temp[index] = arr[left];
```

```
            index++; left++;
```

```
        } else {
```

```
            temp[index] = arr[right];
```

```
            index++; right++;
```

```
        }
```

```
    // left array agar element bache hai.
```

```
    while (left <= mid) {
```

```
        temp[index] = arr[left];
```

```
        index++; left++;
```

```
    // right array me bache ho toh
```

```
    while (right <= end) {
```

```
        temp[index] = arr[right];
```

```
        index++; right++;
```

```
    }
```


For the Cart merging :-

index = 0 ;

while (start <= end) {

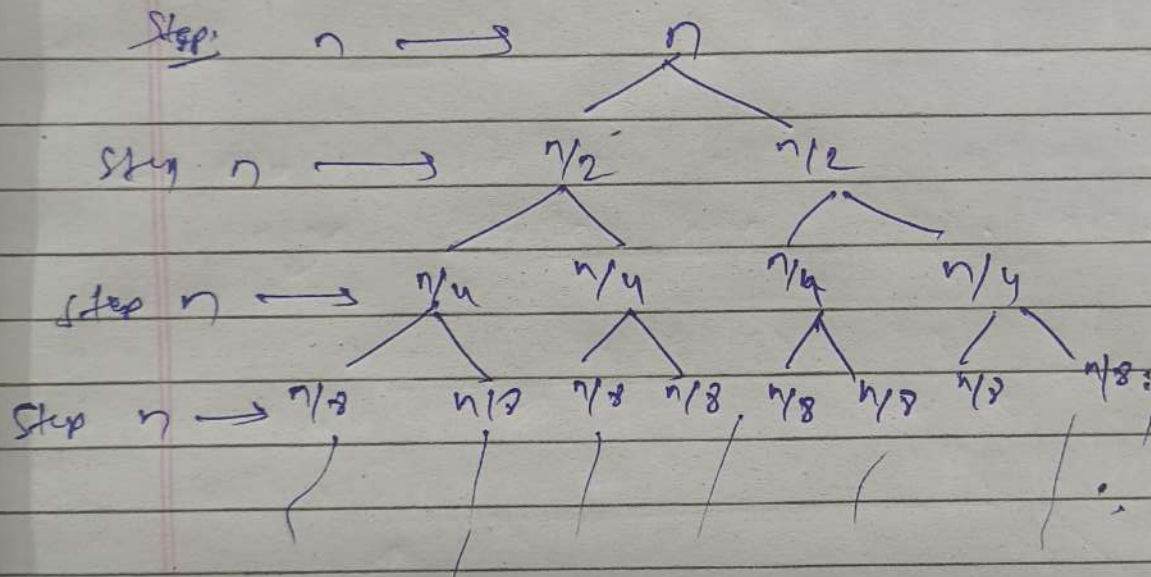
arr[start] = temp[index] ;

start ++ , index ++ ;

}

}

Time Complexity :-



$$\text{Total} = n \ln n \ln n = \log n.$$

$$\therefore \text{Time Complexity} = O(n \log n)$$

$$\text{Space Complexity} = O(n)$$